



CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Which Model, Which Effort

Claude Code deep dive, episode 5

You turned everything up, and the answer got worse.

There are two dials here, not one: the model, and how much it thinks before it answers. Most people know about the first and have never touched the second.

And more is not always better. The documentation warns about its own top effort level in five words, which is the most useful sentence on this page.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- The two dials, and that they interact
- The model aliases as of September 2026
- Which default your plan puts you on
- opusplan, the hybrid almost nobody mentions
- The five effort levels and the warning on the top one
- Where the dials live, and which setting wins
- How to choose from the shape of the task

Out of scope

- Benchmarks and leaderboards: none are quoted here
- /fast, which was not confirmed in public documentation during this check
- Per-token pricing, which is episode 1
- Any claim that one model is better than another in general

This sheet teaches a decision rule rather than a ranking, because a ranking is wrong within a release and a rule is not.

01 Effort levels, September 2026

low	short, scoped, not intelligence sensitive
medium	cost sensitive, trades some intelligence
high	THE DEFAULT on almost everything
xhigh	deeper reasoning, higher token spend
max	demanding tasks, PRONE TO OVERTHINKING
ultracode	xhigh plus dynamic workflow orchestration

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 Two dials, not one

The MODEL is which brain does the work. The EFFORT level is how much it thinks before it answers.

They interact, so the same model at two different effort levels behaves like two different tools.

If you have only ever changed the model, you have been operating half the controls, and the half you were missing decides how long it thinks about your problem.

Effort controls adaptive reasoning: how much the model thinks before responding. Lower effort is faster for straightforward tasks; higher effort provides deeper reasoning for complex problems.

Claude Code docs, Model configuration, checked September 2026

04 The aliases, and why it is not a ladder

haiku for simple fast tasks. sonnet for daily coding. opus for complex reasoning. fable for the hardest and longest running. best resolves to whatever the strongest available one is.

And opusplan, which gets its own section below.

Notice what that list is not. It is not a quality ranking where you should always reach for the top. It is a list of different SHAPES of job.

Aliases include haiku for simple fast tasks, sonnet for daily coding, opus for complex reasoning, fable for the hardest and longest-running tasks, and best for the strongest available.

Claude Code docs, Model configuration, checked September 2026

Switch models with /model during a session, or start with `claude --model <name>`.

Claude Code docs, How Claude Code works, checked September 2026

05 Which one are you on right now

It depends on what you pay for, and almost nobody checks.

On Max, Team Premium, Enterprise and the API, the default is Opus 5. On Pro and Team Standard, it is Sonnet 5.

So two developers on one team, doing the same work, can be having genuinely different experiences and neither knows why. If you have read someone saying this tool is brilliant, or useless, and your experience did not match, you may not have been using the same thing.

The default model is Opus 5 on Max, Team Premium, Enterprise and API accounts, and Sonnet 5 on Pro and Team Standard.

Claude Code docs, Model configuration, checked September 2026



06 The setting nobody mentions

opusplan is a hybrid: Opus for PLANNING and Sonnet for EXECUTION.

The expensive, careful reasoning goes where the leverage actually is, which is deciding what to do. The fast, cheaper model does the part where the decisions are already made.

That is exactly how you would staff a team, and here it is one setting. If you try one thing from this sheet, try that.

The opusplan alias is a hybrid that uses Opus for planning and Sonnet for execution.

Claude Code docs, Model configuration, checked September 2026

07 The documentation warns about its own top setting

On the maximum effort level, the guidance is: demanding tasks, but prone to overthinking, test before adopting broadly.

That is a vendor telling you that more reasoning is not monotonically better, which is rare and worth trusting.

There is a point where extra reasoning stops adding accuracy and starts adding doubt. Both dials at the top on a task that did not need it is paying more for a worse answer, more slowly.

On the max effort level the guidance is: demanding tasks, but prone to overthinking, test before adopting broadly.

Claude Code docs, Model configuration, checked September 2026

The high effort level is the default on all models except one, and balances token usage against intelligence.

Claude Code docs, Model configuration, checked September 2026



08 Why your setting would not stick

Both dials can be set in several places, and the precedence explains the setting that was being ignored.

Environment variables take precedence over all settings. Command line flags apply to that session only. The settings file holds saved defaults, including a different effort per model. And the in-session commands change it now.

If a value in a file is being ignored, look for the variable before you look for a bug. And pressing `s` in either picker applies the change to this session only rather than saving it.

The environment variable for effort level takes precedence over all settings files.

Claude Code docs, Model configuration, checked September 2026

Pressing `s` in the picker applies the change for the current session only, while `Enter` saves it as the default.

Claude Code docs, Model configuration, checked September 2026

09 Your prompting should change when your model does

The documented guidance for the strongest tier is counterintuitive enough to be worth reading twice.

Describe the OUTCOME, not the steps. Hand it ambiguous problems: root cause investigations, outage debugging, architecture decisions. SKIP THE VERIFICATION REMINDERS, because it verifies its own work with less prompting. And size up larger tasks rather than breaking them into pieces.

That third one contradicts a habit most careful engineers have built. "Make sure you run the tests" is good practice on a smaller model and noise on a bigger one.

Guidance for the strongest tier: describe the outcome rather than the steps, hand it ambiguous problems, skip the verification reminders because it verifies its own work with less prompting, and size up larger tasks.

Claude Code docs, Model configuration, checked September 2026

Choosing, without a table

1. Is the hard part DECIDING, or carrying out a decision already made?
2. Is the problem ambiguous, or specified?
3. How long will this run?
4. Would opusplan fit: plan strong, execute fast?
5. Which default does your plan actually put you on?
6. Are you reaching for max effort out of caution rather than need?
7. Is an environment variable overriding the setting you just changed?
8. Have you adjusted your PROMPTING for the model you switched to?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

Model configuration

<https://code.claude.com/docs/en/model-config>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.